

Scanned Code Report

AUDITAGENT

Code Info

Auditor Scan

#	Scan ID		Date
	1		January 23, 2026
	Organization		Repository
	chipi-pay		sessions-smart-contract
	Branch		Commit Hash
	main		628a5122...0c4f71aa

Contracts in scope

src/account.cairo src/lib.cairo src/outside_execution.cairo

Code Statistics

	Findings		Contracts Scanned		Lines of Code
	10		3		964

Findings Summary



Total Findings

- High Risk (5)
- Medium Risk (2)
- Low Risk (1)
- Info (0)
- Best Practices (2)

Code Summary

This protocol implements an advanced, upgradeable smart contract account on Starknet, built upon the OpenZeppelin `AccountComponent`. It enhances standard account functionality with two primary features: Session Keys and Outside Execution (meta-transactions).

The Session Key mechanism allows the account owner to grant temporary, permissioned access to dApps or other services. Instead of signing every transaction with the master key, the owner can issue a session key with specific constraints:

- **Time-based Expiration:** Session keys are only valid until a specified timestamp.
- **Usage Limits:** A maximum number of calls can be defined for each session key.
- **Function Scoping:** Access can be restricted to a specific list of function selectors, preventing the session key from performing unauthorized actions.

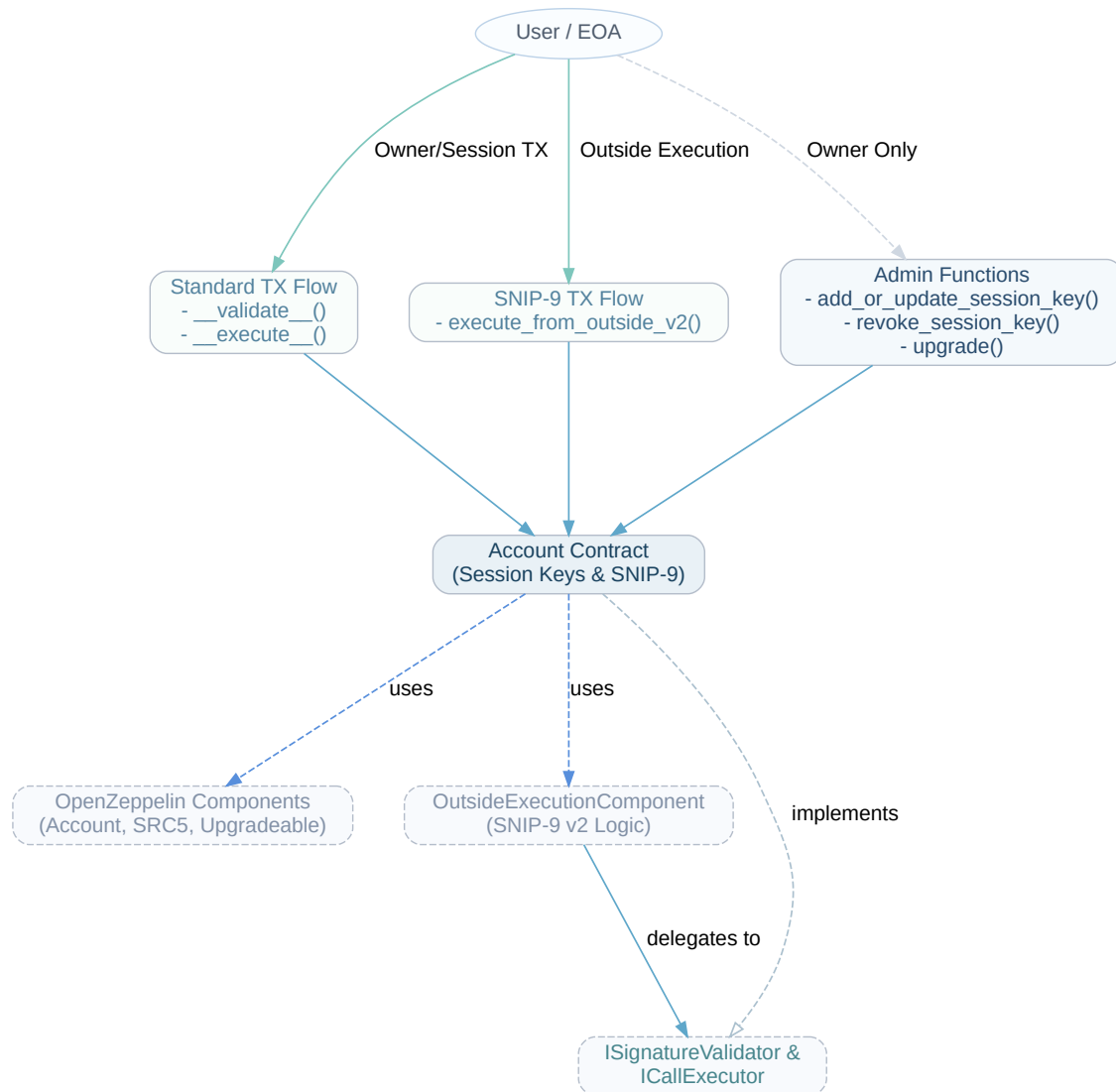
The protocol also implements the SNIP-9 v2 standard for "Outside Execution," enabling meta-transactions. This allows a third party, such as a relayer or paymaster, to submit transactions on behalf of the user. The user signs a structured message containing the transaction details (according to the SNIP-12 typed data hashing standard), and the relayer submits this signed message to the account's `execute_from_outside_v2` endpoint. This feature facilitates gasless transactions for the end-user and improves user experience by abstracting away network fees.

The account's validation logic is customized to handle multiple signature schemes. It can validate standard transactions signed by the owner's master key as well as transactions and messages signed by a valid session key.

Main Entry Points and Actors

- `__execute__`: Executes a series of calls from the account. This is the standard execution entry point.
- **Actors:** Account Owner, Session Key Holder.
- `add_or_update_session_key`: Creates a new session key or updates an existing one with specified permissions (validity period, max calls, allowed functions). This must be called by the account owner.
- **Actors:** Account Owner.
- `revoke_session_key`: Deactivates a session key, immediately revoking its permissions. This must be called by the account owner.
- **Actors:** Account Owner.
- `execute_from_outside_v2`: Executes a bundle of calls defined in a signed `OutsideExecution` message, enabling meta-transactions (SNIP-9 v2).
- **Actors:** Relayer/Executor (submitting a transaction signed by an Account Owner or Session Key Holder).
- `is_valid_signature`: Verifies if a given signature for a given hash is valid according to ERC-1271. This can consume a session call if a session key is used for signing.
- **Actors:** External Verifier (e.g., another smart contract).

Code Diagram



✦ 1 of 10 Findings

src/account.cairo

Unrestricted execute allows anyone to make the account perform arbitrary calls (full fund theft / account takeover)

• High Risk

The account's `__execute__` entrypoint performs arbitrary external calls via `__execute_calls` but does not enforce any caller/authentication restriction.

```
fn __execute__(ref self: ContractState, calls: Array<Call>) -> Array<Span<felt252>> {
    self.__execute_calls(calls)
}

fn __execute_calls(ref self: ContractState, mut calls: Array<Call>) -> Array<Span<felt252>> {
    ...
    match starknet::syscalls::call_contract_syscall(call.to, call.selector, call.calldata)
    {
        Result::Ok(ret) => { res.append(ret); },
        Result::Err(_) => { let mut err = array![]; res.append(err.span()); }
    }
    ...
}
```

On Starknet, `__execute__` is just an external entrypoint like any other: an attacker can send an *Invoke* transaction from their own account contract directly to the victim account's `__execute__` entrypoint. This bypasses `__validate__` entirely (because validation is only executed for the *transaction's sender account*, not for arbitrary contracts it calls).

Concrete exploit scenario (no prerequisites):

1. Victim account holds assets (ERC20/ETH) and/or has allowances/permissions.
2. Attacker submits a normal transaction from their own account calling the victim's `__execute__` with a `calls` array that includes e.g.:
 - `token.transfer(attacker, amount)` (drains ERC20s from victim),
 - `token.approve(attacker, type(uint256).max)` (sets unlimited allowance), and/or
 - a **self-call** to privileged functions guarded by `assert_only_self()`.
3. Because `__execute_calls` executes `call_contract_syscall` from the victim account context, downstream contracts see the victim account as the caller and will honor the transfers/approvals.

Account takeover variant:

- The attacker includes a call where `call.to == victim_account_address` and `call.selector == upgrade(...)` (or `add_or_update_session_key(...)`, `revoke_session_key(...)`). Since the call is executed *from within the account contract*, `get_caller_address()` inside those functions is the account itself, so `assert_only_self()` passes, enabling:
 - upgrading to a malicious class,
 - installing attacker-controlled session keys,
 - revoking legitimate session keys.

Impact:

- Complete loss of all funds held by the account and any approvals the attacker can exploit.
- Full account compromise via upgrade/session management self-calls.

Practical exploitability:

- Immediate and permissionless; does not require any valid owner/session signature, mempool racing, or special timing.

✦ 2 of 10 Findings

src/account.cairo

Session key endpoint whitelist is not enforced in `is_valid_signature()/OutsideExecution` validation, enabling restricted session keys to execute arbitrary calls (fund theft)

• High Risk

Session keys are designed to be function-restricted via `allowed_entrypoints` (enforced in `__validate__` through `is_session_allowed_for_calls`). However, the ERC-1271-like `is_valid_signature()` path (via `_validate_signature_internal`) and the SNIP-9 `execute_from_outside_v2()` path (via `SignatureValidatorImpl::validate_signature`) validate *only* (a) expiry, (b) call quota, and (c) ECDSA correctness, and do **not** enforce the session's `allowed_entrypoints` whitelist.

This means any session key that was intended to be limited (e.g., only a game move function) can still authorize an `OutsideExecution` containing **arbitrary calls**, including token transfers/approvals, as long as the session key signs the `OutsideExecution` hash.

Vulnerable logic (whitelist is ignored):

```
// SignatureValidatorImpl::validate_signature (session path)
if signature.len() == 4 {
    let session_pubkey = *signature.at(0);
    // ... checks: valid_until, session exists, not expired, calls_used < max_calls
    let is_valid = check_ecdsa_signature(hash, session_pubkey, r, s);
    if is_valid {
        self._consume_session_call(session_pubkey);
    }
    return is_valid;
}

// InternalImpl::_validate_signature_internal (session path used by is_valid_signature)
if signature.len() == 4 {
    // ... checks: valid_until, session exists, not expired, calls_used < max_calls
    let ok = check_ecdsa_signature(hash, session_pubkey, r, s);
    if ok {
        self._consume_session_call(session_pubkey);
        return starknet::VALIDATED;
    }
}
```

Intended restriction exists elsewhere but is not applied here:

```
// SRC6Impl::__validate__
if signature.len() == 4 {
    if !self.is_session_allowed_for_calls(session_pubkey, calls.span()) {
        return 0;
    }
    // ... signature check then _consume_session_call
}
```

Concrete exploit scenario (fund theft):

1) Account owner creates a session key with a restrictive whitelist (e.g., only selectors for a specific dApp) and

non-trivial `max_calls`.

2) The dApp/relay (or an attacker who compromises that session private key) crafts an `OutsideExecution` whose `calls` include a malicious transfer/approval from the account (e.g., `ERC20.transfer(attacker, balance)` or `ERC20.approve(attacker, type(uint256).max)`), even though that selector is not whitelisted.

3) The attacker signs the SNIP-9 message hash with the session key and submits `execute_from_outside_v2(outside_execution, [session_pubkey, r, s, valid_until])`.

4) Because `validate_signature` does not call `_is_session_allowed_for_calls` (nor check selectors at all), the signature is accepted and the arbitrary calls are executed, draining assets.

Impact:

- Breaks the protocol's stated security guarantee that session keys are function-restricted.
- Any restricted session key effectively becomes "arbitrary-call capable" under SNIP-9 outside execution, enabling theft of all assets the account can move/approve within the remaining call quota and validity window.

Practical exploitability:

- High: requires only possession/compromise of a session key that was explicitly intended to be low-privilege (a realistic threat model for sessions delegated to external dApps/relayers).

Severity Note:

- A session key with a non-empty `allowed_entrpoints` whitelist exists for the account.
- The holder (or a compromiser) of that session key can submit `execute_from_outside_v2` with arbitrary calls.

✦ 3 of 10 Findings

src/account.cairo

Call-limit bypass vector via calls_used reset on revoke/update (breaks monotonic counter assumption)

• High Risk

Black-box expectation: `calls_used` is monotonically increasing. In code, revocation and updates reset `calls_used` to 0. Alone this may be intended, but when combined with the ability of non-owner session keys to invoke session management (see previous finding), a session can self-reset its counter to bypass `max_calls`.

Relevant code:

```
// add_or_update resets calls_used to 0
fn add_or_update_session_key(...){
    let sess = SessionData { valid_until, max_calls, calls_used: 0, allowed_entrypoints_len:
allowed_entrypoints.len(), };
    self.session_keys.write(session_key, sess);
}

// revoke also zeros calls_used
fn revoke_session_key(...){
    let sess = SessionData { valid_until: 0, max_calls: 0, calls_used: 0,
allowed_entrypoints_len: 0 };
    self.session_keys.write(session_key, sess);
}
```

Impact: If a session key can reach these entrypoints via **execute** (e.g., empty whitelist treated as allow-all), it can loop revoke/update to continually reset `calls_used` and evade `max_calls`, contradicting the black-box invariant and undermining the stated call-limiting guarantee.

Severity Note:

- Owner may configure sessions with `allowed_entrypoints_len == 0` (explicitly treated as allow-all by `_is_session_allowed_for_calls`).
- Self-call via **execute** satisfies `assert_only_self` (as the caller is the account address).

✦ 4 of 10 Findings

src/outside_execution.cairo

Session key endpoint whitelist is not enforced in execute_from_outside_v2, enabling full account privilege escalation (including self-upgrade)

• High Risk

The `execute_from_outside_v2` flow relies on `SignatureValidatorImpl.validate_signature()` to authenticate both owner and session signatures, but the session-signature path does **not** enforce the session key's `allowed_entrypoints` whitelist.

As a result, a session key that is intended to be restricted to specific selectors (e.g., only a game move or ERC20 transfer) can instead authorize **arbitrary calls**, breaking the delegated-permission security model and enabling theft/loss of funds.

Vulnerable logic:

```
// execute_from_outside_v2
let is_valid = contract_state.validate_signature(message_hash, signature);
assert(is_valid, Errors::INVALID_SIGNATURE);

// SignatureValidatorImpl.validate_signature (session path)
if signature.len() == 4 {
    let session_pubkey = *signature.at(0);
    ...
    let session = self.session_keys.read(session_pubkey);
    ...
    // MISSING: _is_session_allowed_for_calls(session_pubkey, outside_execution.calls)
    let is_valid = check_ecdsa_signature(hash, session_pubkey, r, s);
    if is_valid {
        self._consume_session_call(session_pubkey);
    }
    return is_valid;
}
```

Note that `__validate__` correctly checks `_is_session_allowed_for_calls(session_pubkey, calls.span())`, but `execute_from_outside_v2` does not, and `validate_signature()` has no `calls` parameter to enforce selector restrictions.

Concrete exploit scenario (realistic):

- 1) User creates a session key with a restrictive whitelist (e.g., only allowing a dApp-specific selector), and shares that session key with a relayer/dApp expecting limited permissions.
- 2) The relayer/dApp (or anyone who obtains the session private key) signs an `OutsideExecution` that includes unauthorized calls, such as:
 - ERC20 `transfer` from the account to attacker.
 - ERC20 `approve` to attacker/AMM then `transferFrom` drain.
 - Self-call to `upgrade(new_class_hash)` to replace the account code with a malicious implementation.
- 3) The attacker submits `execute_from_outside_v2` with that 4-element session signature.
- 4) Because the whitelist is not checked, the account executes the arbitrary call batch.

Impact:

- Breaks the core guarantee: "No privilege escalation: Sessions can only call whitelisted functions".
- Enables direct asset theft from the account (tokens/NFTs held by the account can be transferred/approved).

- Enables permanent takeover via self-upgrade (the `upgrade` entrypoint is `assert_only_self()`, but when the account executes a call to itself via `call_contract_syscall`, the caller is the account itself, satisfying the check).

Practical exploitability:

- High. Any party with the session key (which is explicitly designed to be delegated to third parties/relayers for gasless UX) can exceed granted permissions using `execute_from_outside_v2`.

Severity Note:

- ERC20 implementations use `msg.sender` for authorization so account-originating calls can move its funds.
- Self-calls through `execute_calls` satisfy `only_self` style checks for upgrade in typical account patterns.

✦ 5 of 10 Findings

src/outside_execution.cairo

Session keys are treated as full ERC-1271 signers (is_valid_signature accepts 4-element session signatures for arbitrary hashes), enabling signature-based privilege escalation outside the call-whitelist model

• High Risk

The account exposes an ERC-1271-style entrypoint `is_valid_signature(hash, signature)` which delegates to `_validate_signature_internal`. That internal helper explicitly accepts **session signatures** (4 elements) and validates them against an **arbitrary hash** without any notion of call context / selector whitelist.

Because ERC-1271 verification is frequently used by external protocols to authorize token movements (orders, permits, withdrawals, etc.), this makes any delegated session key effectively a **general-purpose signer** for the account in those integrations, bypassing the intended “session keys are limited to whitelisted entrypoints” security model.

Vulnerable logic:

```

#[external(v0)]
fn is_valid_signature(
    ref self: ContractState,
    hash: felt252,
    signature: Array<felt252>
) -> felt252 {
    self._validate_signature_internal(hash, signature)
}

fn _validate_signature_internal(
    ref self: ContractState,
    hash: felt252,
    signature: Array<felt252>
) -> felt252 {
    // ...
    if signature.len() == 4 {
        let session_pubkey = *signature.at(0);
        let r = *signature.at(1);
        let s = *signature.at(2);
        let valid_until: u64 = (*signature.at(3)).try_into().unwrap();

        // Only time/quota checks; NO whitelist/call-context enforcement possible here
        let session = self.session_keys.read(session_pubkey);
        if session.valid_until == 0 { return 0; }
        if get_block_timestamp() > session.valid_until { return 0; }
        if session.calls_used >= session.max_calls { return 0; }

        let ok = check_ecdsa_signature(hash, session_pubkey, r, s);
        if ok {
            self._consume_session_call(session_pubkey);
            return starknet::VALIDATED;
        }
        return 0;
    }
    // ...
}

```

Concrete exploit scenario:

- 1) A user creates a session key intended to be narrowly scoped (e.g., for a game or relayer), expecting it cannot authorize arbitrary actions.
- 2) The dApp/relayer (or an attacker who obtains the session private key) produces a session signature over a hash required by some external protocol that relies on `is_valid_signature` (ERC-1271-style) to authenticate the account (e.g., signed orders/permits/withdrawals).
- 3) The external protocol calls `Account.is_valid_signature(order_or_permit_hash, session_signature)` and receives `VALIDATED`, treating the session key as an authorized account signer.
- 4) The external protocol then executes the signature-authorized action, potentially resulting in theft/loss of the account's assets (depending on that protocol's semantics), despite the session key never being intended to have such broad authority.

Impact:

- Session keys can authorize actions in *any* ERC-1271-integrated protocol that uses this account for signature checks, which can bypass the intended session permission model (whitelisted entrypoints), potentially enabling loss of funds via signature-based token approvals/orders/withdrawals.

- This risk exists even if on-chain call-whitelisting is correctly enforced elsewhere, because `is_valid_signature` has no call context to constrain.

Severity Note:

- At least some external protocols will call `is_valid_signature` and execute actions (e.g., token permits, orders, withdrawals) on success.
- Users will issue session keys expecting selector-whitelisting to constrain their authority.

✦ 6 of 10 Findings

src/account.cairo

State Modification in `is_valid_signature` Violates ERC-1271 Standard

• Medium Risk

The external function `is_valid_signature` is designed for compatibility with the ERC-1271 standard, which is widely used by dApps to verify off-chain signatures without requiring a state change. However, the implementation for session key signatures (when `signature.len() == 4`) calls the internal function `_consume_session_call`, which increments the `calls_used` counter for the session key. This state modification in a function expected to be read-only can lead to several critical issues:

1. **Denial of Service:** A dApp or a malicious actor can repeatedly call `is_valid_signature` for an off-chain message (e.g., a marketplace order), consuming the user's limited session calls and rendering the session key useless before it's ever used for an actual transaction.
2. **Broken Integrations:** Protocols that rely on ERC-1271 will behave unexpectedly, as they do not anticipate a state change during a signature check.
3. **Wasted Resources:** Each call to `is_valid_signature` with a valid session signature permanently consumes one of the `max_calls`, even if the underlying action is never executed on-chain.

Code snippet:

```
// File: src/account.cairo

#[external(v0)]
fn is_valid_signature(
    ref self: ContractState,
    hash: felt252,
    signature: Array<felt252>
) -> felt252 {
    self._validate_signature_internal(hash, signature)
}

// ...

fn _validate_signature_internal(
    ref self: ContractState,
    hash: felt252,
    signature: Array<felt252>
) -> felt252 {
    // ...
    // Session path: 4-element signature [session_pubkey, r, s, valid_until]
    if signature.len() == 4 {
        // ... validation checks ...
        let ok = check_ecdsa_signature(hash, session_pubkey, r, s);
        if ok {
            self._consume_session_call(session_pubkey); // <-- State modification
            return starknet::VALIDATED;
        } else {
            return 0;
        }
    }
    // ...
}
```

Severity Note:

- At least one valid session signature becomes publicly observable (e.g., via a prior transaction using the session) before its valid_until.
- The session's valid_until is sufficiently in the future to allow multiple draining calls.
- The attacker is willing to pay transaction fees to grief.

7 of 10 Findings

src/account.cairo

Non-atomic multicall execution swallows subcall failures, enabling partial state changes and fund loss (e.g., approvals/transfers persist even when later calls fail)

• Medium Risk

The account's execution engine intentionally *does not revert* when an individual subcall fails. Instead, it appends an empty return-data span and continues executing subsequent calls. This breaks the common/expected "all-or-nothing" (atomic) behavior of batched operations and can directly lead to irreversible loss of funds (e.g., transfers executed without receiving expected proceeds, or approvals persisting after a failing swap).

Vulnerable logic:

```
fn _execute_calls(ref self: ContractState, mut calls: Array<Call>) -> Array<Span<felt252>> {
    let mut res = array![];
    loop {
        match calls.pop_front() {
            Option::Some(call) => {
                match starknet::syscalls::call_contract_syscall(
                    call.to, call.selector, call.calldata
                ) {
                    Result::Ok(ret) => {
                        res.append(ret);
                    },
                    Result::Err(_) => {
                        let mut err = array![];
                        res.append(err.span());
                    }
                }
            },
            Option::None => { break; },
        }
    }
    res
}
```

Why this is exploitable / high impact:

- Many safe transaction patterns rely on atomicity (if any step fails, earlier steps revert). Here, earlier successful calls are *not reverted* when a later call fails.

- Example realistic loss scenario (swap / aggregator flow):

- 1) A user signs a batched transaction that first performs a token `transfer` or `approve`, then performs a `swap` call.
- 2) The swap call fails (legitimately due to slippage/state changes, or maliciously by the counterparty reverting).
- 3) The account transaction still completes successfully from the account's perspective (no revert), so the earlier `transfer` or `approve` remains in effect.
- 4) Funds may already be moved (if using `transfer`-then-swap patterns), or approvals remain outstanding (if using approve-then-swap patterns), enabling the approved spender/contract to later drain tokens via `transferFrom`.

This issue is amplified by the validation flow for session signatures: successful signature validation increments `session_keys[session_pubkey].calls_used` before execution. Even if later calls fail, the session quota is still consumed, reducing the victim's ability to react/undo/retry operations within the same session.

Net effect: batched operations can silently become “partial commits”, enabling direct token loss or lingering approvals that can be exploited later, contrary to standard user and integrator expectations for account multicalls.

Severity Note:

- Integrators or users expect atomic multicall semantics and design batches accordingly (approve/transfer followed by a call that can fail).
- Wallets/UIs do not consistently surface per-subcall success/failure to prevent unsafe sequencing.
- A malicious or compromised counterparty can cause the later call to revert or rely on natural failures (e.g., slippage).

✦ 8 of 10 Findings

src/account.cairo

Session key can revoke session keys due to assert_only_self + empty-whitelist semantics (owner-only expectation broken)

• Low Risk

Black-box expectation: only the account owner may revoke sessions. In code, `revoke_session_key` is gated by `assert_only_self` (`caller == contract`), not by owner signature. A session-signed tx routed via **execute** is seen as a self-call, and if the session whitelist is empty (treated as allow-all) or explicitly includes the selector, the session can revoke.

Relevant code:

```
// revoke gated only by self-call, not owner-only
fn revoke_session_key(ref self: ContractState, session_key: felt252) {
    self.account.assert_only_self();
    // ...
}

// __validate__ allows 4-elt session signatures when selectors are permitted
if signature.len() == 4 {
    // allow if session permits the calls
    if !self._is_session_allowed_for_calls(session_pubkey, calls.span()) { return 0; }
    // ... on success
    if check_ecdsa_signature(msg_hash, session_pubkey, r, s) {
        self._consume_session_call(session_pubkey);
        return starknet::VALIDATED;
    }
}

// empty whitelist means ALL selectors are allowed
fn _is_session_allowed_for_calls(self: @ContractState, session_key: felt252, calls:
Span<Call>) -> bool {
    let session = self.session_keys.read(session_key);
    if session.allowed_entrypoints_len == 0 { return true; }
    // ...
}
```

Impact: A session created with an empty `allowed_entrypoints` array (or one that includes `revoke_session_key`) can successfully route a call to `revoke_session_key` via **execute**, violating the black-box invariant that revocation requires the owner. This enables unauthorized revocation of other sessions (griefing/DoS against intended delegates).

✦ 9 of 10 Findings

src/account.cairo

Denial of Service via Unsafe Type Conversion in Validation Logic

• Best Practices

The contract is vulnerable to a Denial of Service attack due to an unsafe `unwrap()` on a `felt252` to `u64` conversion in the signature validation logic. In several validation paths (`__validate__`, `validate_signature` for outside execution, and `is_valid_signature`), the contract handles session key signatures which are expected to be a 4-element array `[session_pubkey, r, s, valid_until]`. The fourth element, `valid_until`, is read as a `felt252` and then converted to a `u64` using `(*signature.at(3)).try_into().unwrap()`.

A `felt252` can represent values far larger than the maximum value of a `u64`. If a malicious user submits a transaction with a signature array where the fourth element is a `felt252` value greater than `u64::max`, the `try_into()` method will return `None`, and the subsequent `unwrap()` will cause the contract to panic.

Since this panic occurs within the `__validate__` function, it will cause any transaction attempting to interact with the account to fail validation. This effectively freezes the account, preventing the owner from sending any transactions until the malicious transaction is removed from the mempool.

Code snippet:

```
// File: src/account.cairo

fn __validate__(ref self: ContractState, calls: Array<Call>) -> felt252 {
    // ...
    if signature.len() == 4 {
        let session_pubkey = *signature.at(0);
        let r = *signature.at(1);
        let s = *signature.at(2);
        let valid_until: u64 = (*signature.at(3)).try_into().unwrap(); // <-- Vulnerable
line

        // Check timestamp expiration
        if get_block_timestamp() > valid_until {
            return 0;
        }
        // ...
    }
    // ...
}
```

This vulnerability is also present in `_validate_signature_internal` and the `validate_signature` implementation for the `ISignatureValidator` trait.

✦ 10 of 10 Findings

src/account.cairo

Revoking a session key may leave stale session_entrypoints in storage when the session was previously updated to a smaller whitelist

• Best Practices

`add_or_update_session_key` overwrites the `SessionData.allowed_entrypoints_len` and writes new entrypoints only for indices `[0..new_len-1]`, but it does not clear any previously-stored entrypoints at indices `>= new_len`.

On revocation, `revoke_session_key` clears entrypoints only up to the *current* `allowed_entrypoints_len`:

```
let current_session = self.session_keys.read(session_key);
let entrypoints_to_clear = current_session.allowed_entrypoints_len;
...
if i >= entrypoints_to_clear { break; }
self.session_entrypoints.write((session_key, i), 0);
```

Therefore, if a session key was once set with a longer whitelist and later updated to a shorter whitelist, revocation will only clear the shorter range and any older entrypoints beyond that range remain non-zero in `session_entrypoints`.

While `_is_session_allowed_for_calls` iterates only up to `allowed_entrypoints_len` (so these stale values are not consulted for authorization as long as lengths are consistent), this behavior breaks the documented/assumed property that “after revocation, a session record is fully zeroed and no entrypoints remain,” and can lead to misleading state for off-chain indexers or on-chain readers that query `get_session_allowed_entrypoint_at` beyond the current length.

Disclaimer

Kindly note, no guarantee is being given as to the accuracy and/or completeness of any of the outputs the AuditAgent may generate, including without limitation this Report. The results set out in this Report may not be complete nor inclusive of all vulnerabilities. The AuditAgent is provided on an 'as is' basis, without warranties or conditions of any kind, either express or implied, including without limitation as to the outputs of the code scan and the security of any smart contract verified using the AuditAgent.

Blockchain technology remains under development and is subject to unknown risks and flaws. This Report does not indicate the endorsement of any particular project or team, nor guarantee its security. Neither you nor any third party should rely on this Report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset.

To the fullest extent permitted by law, Nethermind disclaims any liability in connection with this Report, its content, and any related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. Nethermind does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party through the product, any open source or third-party software, code, libraries, materials, or information linked to, called by, referenced by or accessible through the report, its content, and the related services and products, any hyperlinked websites, any websites or mobile applications appearing on any advertising, and Nethermind will not be a party to or in any way be responsible for monitoring any transaction between you and any third-party providers of products or services. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE.